

Homework 4: Booleans and Conditionals

- Due Sunday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jun 11 at 12am - Jun 26 at 11:59pm

Learning Objectives

- Use Lists, Strings and Dictionaries
- Practice formatted output.
- Use if conditions in programming.
- Use if conditions with complex conditions.
- Use nested if statements.

Things to consider

1. Submit the solutions to all the problems on gradescope. Files to be submitted are: Hw04.py, and Reflection.pdf. Submission of pdf files is not enforced in the grader.
2. The output of your program must match the examples exactly for points.
3. This is an individual assignment. Do not use AI or ask your friends for solutions. We will be running similarity checks in gradescope.
4. Run given tests locally before submitting. It will save you a lot of time. Local runs are a lot faster than gradescope submissions.

Q1: Dictionary For data storage

Write a function `receipt_entry_and_print()` (in Hw04.py) that produces a grocery store checkout receipt. Because we have not learned about loops yet we will limit ourselves to exactly 4 items so imagine that your software is running on a checkout counter that says “4 items only”. Prompt the user for 4 items. The user will enter three things for each item separated by spaces.

1. Name of the item
2. Price per unit
3. Quantity



For example: input “apples 1.80 2.5” mean item name is “apples”, price per unit is \$1.80/lb and 2.5lb apples are purchased.

You must accept the user input and store it in a dictionary. The keys for the dictionary will be 1, 2, 3 and 4 and the value will be tuples. For example, the tuple for the item entered above is (“apples”, 1.80, 2.5).

Print the dictionary after all 4 items are added. You should be able to do this by passing the constructed dictionary to the print function. Then print the receipt in the format shown below. You will need format control strings for this. You need to calculate the required totals. The field sizes are 3, 14, 14, 12, and 9. **You should create functions to help reduce the repetition in this question.**

Example:

```
Enter item : Apples 1.8 2.5
Enter item : Chicken 3.9 3
Enter item : Bread 1.88 2
Enter item : Milk 2.59 1
```

```
Data as dictionary : {1: ('Apples', 1.8, 2.5), 2: ('Chicken', 3.9, 3.0), 3: ('Bread', 1.88, 2.0), 4: ('Milk', 2.59, 1.0)}
```

No.	Item	Unit Price	Quantity	Total
1	Apples	1.80	2.50	4.50
2	Chicken	3.90	3.00	11.70
3	Bread	1.88	2.00	3.76
4	Milk	2.59	1.00	2.59
Total				22.55

Q2: Shapes

Write a function `shapes()` (Hw04.py), that asks the user to select a shape (circle, triangle, square and rectangle). If the input is not a valid shape then the program must terminate with the message “Invalid shape”. Once the shape is selected the user is prompted for the appropriate dimensions. For circle and square, only one dimension is needed (radius for circle, side for square). For a rectangle, you need to ask for the length of two sides. For a triangle, you need to ask for 3 sides. You may use $\pi = 3.141592653$ for this lab.



For the area of a triangle, you can use Heron's formula:

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)} \quad \text{where } s = \frac{a+b+c}{2}$$

The final answers must be rounded and displayed to 3 decimal places.

Example 1: `shapes()`

```
Select shape (circle, square, triangle, rectangle) : circle
Enter radius = 7
Area of circle = 153.938
```

Example 2: `shapes()`

```
Select shape (circle, square, triangle, rectangle) : pentagon
Invalid shape
```

Example 3: `shapes()`

```
Select shape (circle, square, triangle, rectangle) : square
Enter side : 7
Area of square = 49.000
```

Example 4:

```
Select shape (circle, square, triangle, rectangle) : rectangle
Enter two sides : 7 8
Area of rectangle = 56.000
```

Example 5:

```
Select shape (circle, square, triangle, rectangle) : triangle
Enter three sides : 3.00 4.00 5.00
Area of triangle = 6.000
```

Question 3: The Three-Word Analyzer

Write a function named `analyze_three_words(word_list)` (in Hw04.py) that takes a list containing exactly three strings. Since you cannot use loops, you will extract each word using its list index (`0`, `1`, and `2`). Your function must analyze and modify each word based on specific rules, then print a formatted report.

1. For each of the three list elements, apply the following rules:

- Rule 1: If a word has exactly 3 characters and starts with the letter 't' or 'T', change that element so the word is completely uppercase.
- Rule 2: If a word is longer than 5 characters and contains the letter 'e', change that element by using string replacement to swap all lowercase 'e's with an asterisk (`*`).
- Once again, you are welcome to define functions to reduce repetition but you are not



allowed to use loops.

2. You do not need to return anything from the function. The changes to the list would be visible to the caller.

Test Script

```
# Test Case 1: Hits Rule 1 ("The"), Rule 2 ("elephant"), and Neither ("cat")
list_1 = ["The", "elephant", "cat"]
# Test Case 2: Hits Rule 1 twice ("tea", "Ten"), Neither ("dog")
list_2 = ["tea", "dog", "Ten"]
# Test Case 3: Hits Rule 2 twice ("engineer", "awesome"), Neither ("bird")
list_3 = ["engineer", "awesome", "bird"]

analyze_three_words(list_1)
analyze_three_words(list_2)
analyze_three_words(list_3)

print(list_1)
print(list_2)
print(list_3)
```

Expected Output

```
['THE', '*l*phant', 'cat']
['TEA', 'dog', 'TEN']
['*ngin**r', 'aw*som*', 'bird']
```

Question 4: Theme Park Ride Access Validator

You are programming the safety scanner for a theme park. Write a function named `check_ride_access(rider, ride)` (in `Hw04.py`) that takes two dictionaries: one for a rider's profile and one for the ride's requirements. The result of the function will be a dictionary with two keys `"status"` and `"reason"`. You may find nested if statement helpful for this question.

1. Determine the rider is allowed on the ride using the following rules.
 - If the rider's `heart_condition` is `True`, set status to `"Denied"` and reason to `"Safety Risk"` and return.
 - If the rider does not have a heart condition, compare the rider height against the ride's `min_height`.
 - If the rider's `height_cm` is less than the ride's `min_height`, set status to `"Denied"` and reason to `"Too Short"` and return.
 - If the height requirement is satisfied, then check the ride age restrictions.
 - If the ride `is_extreme` is `True` and the rider's `age` is less than 16, set status to `"Denied"` and reason to `"Too Young for Extreme Rides"` and return.
 - If all the rules are satisfied, then set status to `"Approved"` and reason to `"N/A"` and return.



Test Script

```
# The Ride Data
the_coaster = {"name": "Death Drop", "min_height": 140, "is_extreme": True}

# Rider Data (Testing all logical paths)
# Case 1: Has a heart condition (Denied: Safety Risk)
rider_1 = {"name": "Alex", "age": 25, "height_cm": 180, "has_fast_pass": True, "heart_condi
tion": True}
# Case 2: Too short (Denied: Too Short)
rider_2 = {"name": "Sam", "age": 12, "height_cm": 130, "has_fast_pass": False, "heart_condi
tion": False}
# Case 3: Tall enough, but too young for extreme ride (Denied: Too Young)
rider_3 = {"name": "Jordan", "age": 14, "height_cm": 150, "has_fast_pass": True, "heart_con
dition": False}
# Case 4: Approved, has Fast Pass (Approved, 5 mins)
rider_4 = {"name": "Casey", "age": 18, "height_cm": 160, "has_fast_pass": True, "heart_cond
ition": False}
# Case 5: Approved, no Fast Pass (Approved, 45 mins)
rider_5 = {"name": "Taylor", "age": 30, "height_cm": 175, "has_fast_pass": False, "heart_co
ndition": False}

# Call the function sequentially
print(check_ride_access(rider_1, the_coaster))
print(check_ride_access(rider_2, the_coaster))
print(check_ride_access(rider_3, the_coaster))
print(check_ride_access(rider_4, the_coaster))
print(check_ride_access(rider_5, the_coaster))
```

Expected Output:

```
{'status': 'Denied', 'reason': 'Safety Risk'}
{'status': 'Denied', 'reason': 'Too Short'}
{'status': 'Denied', 'reason': 'Too Young for Extreme'}
{'status': 'Approved', 'reason': 'N/A'}
{'status': 'Approved', 'reason': 'N/A'}
```

Question 5: Recursive String Reversal

Now that you are familiar with functions and `if` conditions, you will implement a recursive function. Write a function named `reverse_string(text)` (in `Hw04.py`) that takes a string as an argument and returns the reversed version of that string using recursion. Because you have not learned loops yet, you must solve this by having the function call itself.

Use the following logic to build your recursive function:

- Base Case: If the length of the `text` is 0 or 1, it is already reversed. Use an `if` statement to check this and simply return the `text`.
- Recursive Step: If the string is longer than 1 character, divide the string into two parts: the first character and the rest of the string. Return the result of calling `reverse_string()` on the rest of the string, concatenated with the first character at the end.



Test Script

```
# Test Cases for Recursive String Reversal
test_1 = "hello"
```

```
test_2 = "recursion"
test_3 = "A"
test_4 = ""

print(reverse_string(test_1))
print(reverse_string(test_2))
print(reverse_string(test_3))
print(reverse_string(test_4)) // This is empty so nothing show up in the output.
```

Expected Output:

```
olleh
noisrucer
A
```

Reflection (5% bonus points for submission)

Reflect on the topics covered this week and write short answers for the questions below. Submit as a pdf called Reflection.pdf.

1. What did we learn this week?
2. Rate you understanding of the topics this week. (1. Did not understand at all, ... 5. Fully understand the topic)
3. Are there any question related to this week's topics? We will address the questions either at the beginning of a lab or through a post.

This tool needs to be loaded in a new browser window

Load Homework 4: Booleans and Conditionals in a new window

